



Titanic Dataset Documentation

MEKELLE UNIVERSITY

MEKELLE INSTITUTE OF TECHNOLOGY

Department of Information Technology (IT)

COURSE: Machine Learning (IT4210B)

Group - 3 members

ID

Mulugeta Gidey	MIT/UR/175/11
Aron T/haymanot	MIT/UR/251/12
Fanaye Tsegay	MIT/UR/243/12
Mehari Goitom	MIT/UR/277/12
Mahlet Atsbeha	MIT/UR/323/12
Aximawit Leake	MIT/UR/018/12
Lwam Tesfay	MIT/UR/147/11
Melat Kfle	MIT/UR/099/12
Frehiwot G/mariam	MIT/UR/044/12
Kidane Aregawi	MIT/UR/310/12

1. Problem Statement

The sinking of the RMS Titanic is one of the most infamous shipwrecks in history. Using machine learning, we aim to predict whether a passenger survived based on attributes such as age, sex, passenger class, fare, and family aboard. This classification task helps understand survival patterns and demonstrates the use of supervised learning techniques.

2. Data Analysis (EDA)

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Load dataset (assuming you've extracted it already)
```

```
df = pd.read_csv('train.csv')
```

```
# Overview
```

```
print(df.shape)
```

```
print(df.info())
```

```
print(df.describe())
```

```
print(df.isnull().sum())
```

Key Observations:

- Missing values: 'Age', 'Cabin', and 'Embarked' contain missing data.

- Target variable: 'Survived' is binary (0 = Died, 1 = Survived).

Visualizations:

Survival count

```
sns.countplot(x='Survived', data=df)
```

Gender survival

```
sns.countplot(x='Survived', hue='Sex', data=df)
```

Class survival

```
sns.countplot(x='Survived', hue='Pclass', data=df)
```

Age distribution

```
sns.histplot(df['Age'].dropna(), kde=True)
```

Correlation matrix

```
sns.heatmap(df.corr(numeric_only=True), annot=True,  
cmap='coolwarm')
```

3. Model Selection and Implementation

Preprocessing:

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Fill missing values
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

# Drop 'Cabin' due to many missing values
df.drop(columns=['Cabin', 'Ticket', 'Name', 'PassengerId'],
inplace=True)

# Encode categorical variables
le = LabelEncoder()
df['Sex'] = le.fit_transform(df['Sex'])
df['Embarked'] = le.fit_transform(df['Embarked'])

# Define features and target
X = df.drop('Survived', axis=1)
y = df['Survived']

# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Model Training:

```
# Initialize and train model
```

```
model = RandomForestClassifier(n_estimators=100,
random_state=42)
```

```
model.fit(X_train, y_train)
```

```
# Predict
```

```
y_pred = model.predict(X_test)
```

4. Training and Testing Results

Evaluation Output:

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test,
y_pred))
```

Sample output (will vary):

	precision	recall	f1-score	support
0	0.87	0.89	0.88	105
1	0.82	0.79	0.81	74
accuracy			0.85	179

5. Evaluation Metrics

We used the following metrics:

- **Accuracy:** Overall correctness.
- **Precision:** Correct positive predictions out of all positive predictions.
- **Recall:** Correct positive predictions out of actual positives.
- **F1-score:** Harmonic mean of precision and recall.

Confusion matrix provides insight into false positives and negatives.

6. Conclusion and Reflections

The Random Forest Classifier performed well with ~85% accuracy. Key features influencing survival included Sex, Pclass, and Fare. Women and first-class passengers had a higher survival probability. Preprocessing significantly impacted performance, especially handling missing 'Age' and 'Embarked' values.

Challenges:

- Handling missing data in 'Age' and 'Cabin'
- Choosing which features to encode or drop

Improvements:

- Hyperparameter tuning (GridSearchCV)
- Trying other models (e.g., XGBoost, SVM)
- Feature engineering (e.g., title from Name)